

Assembly and Low-Level Programming

A complete, interview-focused guide to how code actually runs on the CPU: registers, memory operands, instructions, function calls, stack frames, calling conventions, x86-64 and ARM basics, system calls, disassembly, SIMD, and inline assembly.

1. Overview

Definition

Assembly language is a human-readable form of the machine instructions a CPU executes. Each assembly instruction maps (almost) one-to-one to a binary **machine instruction**. "Low-level programming" means working close to the hardware - directly with **registers** (tiny fast storage inside the CPU), **memory addresses**, and the **instruction set architecture (ISA)** like x86-64 or ARM.

When you write `a = b + c` in C, the compiler turns it into something like:

```
mov  eax, [b]      ; load b from memory into register eax
add  eax, [c]      ; add c to eax
mov  [a], eax      ; store result back to memory
```

Why it matters

- **Performance:** Understanding what the compiler emits lets you write code that runs faster (cache-friendly loops, SIMD, avoiding branches).
- **Debugging:** When you only have a crash address, a core dump, or a stripped binary, assembly is the only truth.
- **Security:** Buffer overflows, ROP chains, shellcode, and exploit mitigation all live at this level.
- **Systems work:** OS kernels, bootloaders, device drivers, JIT compilers, and language runtimes need hand-written assembly or inline assembly.

Where it is used in real systems

Area	Use of assembly / low-level knowledge
Operating systems	Context switches, interrupt handlers, syscall entry, boot code
Compilers / JITs	Code generation, calling conventions, register allocation
Cryptography	Constant-time code, hand-tuned AES/SHA using SIMD/AES-NI
Games / HPC	SIMD vectorization, cache optimization
Security	Exploit dev, malware analysis, reverse engineering
Embedded / firmware	Direct hardware register access, tight memory budgets

Why interviewers ask about it

It separates people who *use* abstractions from people who *understand* them. Questions test whether you know:

- What the stack actually is and how function calls work.
- Why some registers survive a function call and some don't (caller/callee-saved).
- How memory, pointers, and CPU registers relate.
- How to reason about performance and undefined behavior.

Even if you never write assembly at work, the mental model is priceless for debugging, performance, and systems design rounds.

2. Core Idea

Intuition

A CPU is a very fast, very literal machine that can only do tiny steps: *load this number, add these two, compare them, jump if equal, store this back*. It has a small set of **registers** (like a handful of scratchpad variables it can access in ~1 cycle) and access to a huge but slower **memory** (RAM). Everything your program does is a long sequence of these tiny steps.

Real-world analogy

Think of a **chef at a small counter**:

- **Registers** = the few items the chef can hold in their hands and on the small cutting board right now (super fast to use, but only a handful fit).
- **Memory (RAM)** = the pantry across the kitchen. Bigger, but you must walk over (a **load**) to fetch something and walk back (a **store**) to put it away.
- **Instructions** = the recipe steps: "chop this", "mix these two", "if the sauce is too thin, go back three steps".
- **The stack** = a stack of plates the chef uses for temporary work and to remember "where was I before I started this sub-task".

The chef can't cook with ingredients still in the pantry - they must first bring them to the counter (registers). This is the **load/store** model.

Small example

C code:

```
int add(int x, int y) {  
    return x + y;  
}
```

x86-64 assembly (simplified, System V ABI - arguments in **edi**, **esi**):

```

add:
    mov  eax, edi    ; eax = x  (first arg in edi)
    add  eax, esi    ; eax = x + y  (second arg in esi)
    ret              ; return value is in eax

```

Step-by-step explanation

1. The caller places **x** in register **edi** and **y** in **esi** (the calling convention says so).
2. The caller executes **call add**, which pushes the return address onto the stack and jumps to **add**.
3. Inside **add**, we copy **x** into **eax** (the register that holds return values).
4. We add **y** to it.
5. **ret** pops the return address off the stack and jumps back. The caller reads the result from **eax**.

That's the whole machine: move data into registers, compute, move results out, jump around, and use the stack to remember where you came from.

3. Important Subtopics

3.1 Registers and Memory Operands

What it means: Registers are named storage locations inside the CPU (e.g. **rax**, **rbx**, **x0**, **x1**). A *memory operand* is a way to reference data in RAM by computing an address, e.g. **[rbp - 8]** or **[rax + rcx*4 + 16]**.

Why it matters: Registers are ~100x faster than RAM. Good code keeps hot values in registers. Memory operands (with base + index*scale + displacement addressing) are how arrays and structs are accessed.

Example (x86-64 addressing):

```

mov  eax, [rbx + rcx*4 + 8]    ; eax = *(int*)(rbx + rcx*4 + 8)
                                ; e.g. array element: base rbx, index rcx, 4-byte
ints

```

Interview angle: "What is the difference between a register and memory?" and "Explain the addressing mode **[base + index*scale + disp]**."

3.2 Load and Store Instructions

What it means: Instructions that move data between registers and memory.

- **Load** = memory → register (**mov reg, [addr]** on x86; **ldr** on ARM).
- **Store** = register → memory (**mov [addr], reg** on x86; **str** on ARM).

Why it matters: x86 is a *register-memory* architecture (arithmetic can touch memory directly). ARM/RISC-V are *load-store* architectures - arithmetic works **only** on registers, so you must explicitly load, compute, store. This distinction shows up constantly.

Example (ARM64):

```
ldr  x0, [x1]      ; load 64-bit value at address in x1 into x0
add  x0, x0, #1     ; increment
str  x0, [x1]      ; store it back
```

Interview angle: "Why can't ARM add two memory locations directly?" (Because it's load-store; only registers feed the ALU.)

3.3 Arithmetic and Logical Instructions

What it means: Core compute instructions: `add`, `sub`, `imul/mul`, `idiv/div`, and bitwise `and`, `or`, `xor`, `not`, `shl/shr` (shifts), `sar` (arithmetic shift).

Why it matters: These set **flags** (Zero, Sign, Carry, Overflow) that branches depend on. Compilers replace expensive ops with cheap ones (strength reduction): `x * 8` becomes `shl x, 3`; `x % 2` on unsigned becomes `and x, 1`.

Example:

```
xor  eax, eax      ; idiomatic "eax = 0" (shorter/faster than mov eax, 0)
shl  ebx, 2        ; ebx = ebx * 4
and  ecx, 1        ; ecx = ecx & 1 (test lowest bit / mod 2)
```

Interview angle: "Why is `xor eax, eax` used to zero a register?" (Smaller encoding, breaks dependency chains, recognized by the CPU as a zeroing idiom.)

3.4 Branch and Jump Instructions

What it means: Control flow. `jmp` is unconditional. Conditional jumps (`je`, `jne`, `jg`, `jl`, `jle`...) test flags set by a prior `cmp/test`. `call/ret` handle function entry/exit.

Why it matters: Branches implement `if`, loops, and switches. Mispredicted branches cost ~15-20 cycles (pipeline flush), so branchless tricks (`cmov`, bit hacks) matter for performance.

Example (an `if (a > b)`):

```
cmp  eax, ebx      ; compute eax - ebx, set flags
jle  .else         ; if eax <= ebx, jump
; ... then-branch ...
jmp  .end
.else:
; ... else-branch ...
.end:
```

Interview angle: "How does `if/else` compile?" and "What is branch prediction and why does branchless code help?"

3.5 Function Calls

What it means: The `call/ret` mechanism plus the convention for passing arguments and return values.

- `call foo` pushes the **return address** and jumps.
- `ret` pops it and jumps back.
- Arguments go in specific registers (and overflow onto the stack); return value comes back in `rax` (x86-64) / `x0` (ARM64).

Why it matters: This is the backbone of structured programming and where the **stack** and **calling conventions** meet.

Interview angle: "What exactly happens when you call a function?" (Push return address, jump, set up frame, do work, restore, `ret`.)

3.6 Stack Frames

What it means: Each function call gets a region of the stack (its **frame**) holding the return address, saved registers, and local variables. `rbp` (frame/base pointer) often marks the frame's base; `rsp` (stack pointer) marks its top. The stack grows **downward** (toward lower addresses) on x86-64 and ARM.

Typical prologue/epilogue (x86-64):

```
push rbp          ; save caller's frame pointer
mov  rbp, rsp      ; set up our frame pointer
sub  rsp, 32        ; allocate 32 bytes of locals
; ... body, locals at [rbp-8], [rbp-16], ...
mov  rsp, rbp      ; (or leave)
pop  rbp           ; restore caller's frame pointer
ret
```

Why it matters: Understanding frames explains recursion, stack overflow, local variables, and how debuggers unwind a call stack (backtrace).

Interview angle: "Draw the stack frame of a function" or "What causes a stack overflow?" (Unbounded recursion / huge local arrays exhausting the stack region.)

3.7 Calling Conventions

What it means: The contract between caller and callee: which registers hold which arguments, where the return value goes, who cleans up the stack, and alignment rules.

System V AMD64 ABI (Linux/macOS) integer args: `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`, then stack. Return in `rax`.

Windows x64 integer args: `rcx`, `rdx`, `r8`, `r9`, then stack. Return in `rax`. Plus 32 bytes of "shadow space".

Why it matters: Mixing conventions crashes programs. It's essential for FFI (calling C from another language), writing assembly by hand, and reverse engineering.

Interview angle: "What is a calling convention and why do we need a standard one?" (So separately-compiled code and libraries interoperate.)

3.8 Caller-Saved vs Callee-Saved Registers

What it means:

- **Caller-saved (volatile/scratch):** The callee may freely overwrite these. If the caller needs the value after the call, *the caller* must save/restore it. (x86-64 SysV: `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8-r11`.)
- **Callee-saved (non-volatile/preserved):** The callee must restore these to their original value before returning. (x86-64 SysV: `rbx`, `rbp`, `r12-r15`, and `rsp`.)

Why it matters: It's a negotiation to minimize saves. Frequently reused values live in callee-saved registers; short-lived scratch values live in caller-saved ones.

Example: If your function calls another function inside a loop and wants to keep a counter in a register across the call, put it in a **callee-saved** register (like `rbx`) so it survives.

Interview angle: Extremely common: "Difference between caller-saved and callee-saved registers?" See the comparison table in Section 8.

3.9 x86-64 Assembly Basics

What it means: The 64-bit extension of Intel/AMD's x86. 16 general-purpose 64-bit registers (`rax..r15`), variable-length CISC instructions, register-memory operations, rich addressing modes. Two syntaxes: **Intel** (`mov dst, src`) and **AT&T** (`movq %src, %dst`).

Register name sizes: `rax` (64-bit) / `eax` (32) / `ax` (16) / `al` (8). Writing to a 32-bit register (`eax`) **zeroes** the upper 32 bits of `rax`.

Interview angle: "Name some x86-64 registers and their roles" (`rsp`=stack ptr, `rbp`=base ptr, `rax`=return, `rip`=instruction ptr).

3.10 ARM Assembly Basics

What it means: ARM (AArch64/ARM64) is a RISC, **load-store** architecture. 31 general 64-bit registers (`x0..x30`, `w0..w30` for 32-bit views), fixed 4-byte instructions, dedicated link register `x30` (LR) for return addresses, `sp` and `pc`.

Key differences from x86:

- Arithmetic only on registers (must `ldr/str` for memory).
- `bl` (branch-with-link) saves the return address in LR instead of pushing to the stack.
- Fixed-length instructions → simpler decoding, easier pipelining.

Example:

```
add x0, x0, x1    ; x0 = x0 + x1    (registers only)
bl  foo           ; call foo, return address saved in x30 (LR)
ret               ; return to address in LR
```

Interview angle: "Compare RISC vs CISC" and "How does ARM return from a function without touching the stack?" (Via the link register.)

3.11 System Call Interface

What it means: The controlled doorway from user space (your program) into the OS kernel to request privileged operations (read a file, allocate memory, send a network packet). You put a **syscall number** in a register, arguments in others, and execute a special instruction (**syscall** on x86-64 Linux, **svc #0** on ARM64).

Example (Linux x86-64, write "Hi\n" to stdout):

```
mov rax, 1        ; syscall number 1 = write
mov rdi, 1        ; fd 1 = stdout
lea rsi, [msg]    ; buffer pointer
mov rdx, 3        ; length
syscall           ; trap into kernel
```

Why it matters: Every I/O, process, and memory operation ultimately becomes a syscall. It's the user/kernel boundary and a key security surface.

Interview angle: "What happens on a system call?" (Switch from user mode to kernel mode, kernel validates & performs the request, returns.)

3.12 Disassembling Compiled Programs

What it means: Turning a compiled binary back into readable assembly to understand or audit it. Tools: **objdump -d**, **gdb**, **radare2/rizin**, **Ghidra**, **IDA Pro**. On Godbolt (Compiler Explorer) you can see the assembly for any C/C++/Rust snippet live.

Example:

```
gcc -O2 -c add.c -o add.o
objdump -d add.o      # show disassembly
gcc -S -O2 add.c      # emit .s assembly directly
```

Why it matters: Reverse engineering, debugging optimized code, security analysis, verifying the compiler did what you expect.

Interview angle: "How would you inspect what a compiler generated?" (`gcc -S, objdump -d`, Godbolt.)

3.13 SIMD Instructions

What it means: Single Instruction, Multiple Data. One instruction operates on a *vector* of values at once - e.g. add 8 pairs of 32-bit floats in one op. x86: SSE, AVX, AVX-512 (registers `xmm`, `ymm`, `zmm`). ARM: NEON, SVE.

Why it matters: Massive throughput for data-parallel work: image/audio/video processing, ML, crypto, string search (`memchr`, JSON parsing). This is "data parallelism" inside a single core.

Example (AVX, add 8 floats):

```
vaddps ymm0, ymm1, ymm2 ; ymm0[i] = ymm1[i] + ymm2[i] for 8 lanes
```

Or in C with intrinsics:

```
__m256 c = _mm256_add_ps(a, b); // 8 floats added at once
```

Interview angle: "What is SIMD and where is it used?" and "How does the compiler auto-vectorize a loop?"

3.14 Inline Assembly

What it means: Embedding assembly directly inside a higher-level language (usually C/C++) using `asm / __asm__`. Lets you use instructions the compiler won't emit (special CPU instructions, `rdtsc`, atomic primitives, syscalls).

Example (GCC extended asm, read timestamp counter):

```
static inline unsigned long long rdtsc(void) {
    unsigned int lo, hi;
    __asm__ volatile ("rdtsc" : "=a"(lo), "=d"(hi));
    return ((unsigned long long)hi << 32) | lo;
}
```

The `"=a"` and `"=d"` are **constraints** telling the compiler outputs land in `eax/edx`.

Why it matters: Needed for kernels, drivers, cryptography, and performance hotspots. Also dangerous: you must correctly declare inputs, outputs, and **clobbers** or the compiler will miscompile around it.

Interview angle: "When would you use inline assembly, and what are the risks?" (Use for what the compiler can't express; risk is mismatched clobber lists corrupting registers.)

4. Real-World Example

Where this all shows up: a `printf("Hello\n")` call in a normal C program.

1. **Function call + calling convention:** `printf`'s format string pointer goes into `rdi`, and `call printf` pushes the return address.
2. **Stack frame:** `printf` sets up its frame, allocates locals for formatting.
3. **Load/store + arithmetic:** It parses the format string byte by byte (loads), computes lengths.
4. **System call:** Ultimately `printf` calls `write(1, buffer, len)`, which becomes a `syscall` that traps into the **kernel**.
5. **Kernel side:** The OS switches to kernel mode, validates the buffer pointer, copies bytes to the terminal driver, returns.
6. **Return:** `ret` unwinds back through `printf` to your `main`, restoring **callee-saved** registers along the way.

Other concrete places:

- **Operating system:** Context switch saves all registers of the outgoing thread to its stack and loads the next thread's - pure assembly.
- **Database:** Hot inner loops (e.g. columnar scans, hashing) are SIMD-vectorized for throughput.
- **Browser (V8/JS engines):** The JIT compiler emits x86-64/ARM machine code at runtime respecting the platform calling convention.
- **Backend server:** A stack trace in a crash dump is read by unwinding stack frames using saved `rbp`/return addresses.
- **Security:** A buffer overflow overwrites the saved return address on the stack to hijack control flow.

5. Diagrams / Mental Models

The stack during a call (x86-64, grows downward)

```

Higher addresses
+-----+
| caller's locals |
+-----+
| argument 7, 8, ... | <- extra args passed on stack
+-----+
| return address | <- pushed by `call`
+-----+ <- rbp points here-ish after prologue
| saved rbp (caller's) |
+-----+
| callee local var 1 | [rbp - 8]
| callee local var 2 | [rbp - 16]
+-----+ <- rsp (top of stack)
Lower addresses

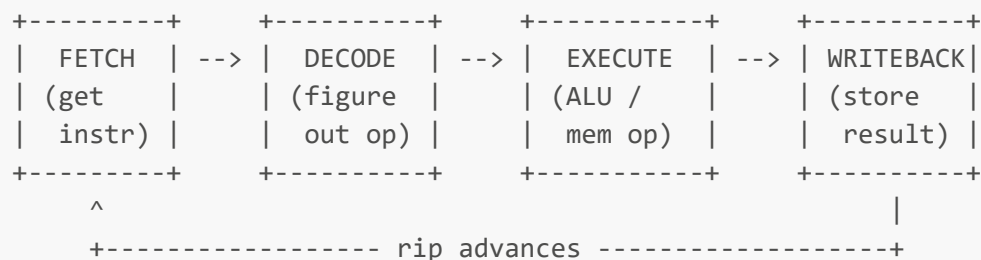
```

Register / memory speed hierarchy

CPU Registers	~1 cycle	(dozens of bytes)	fastest

L1 Cache	~4 cycles	(~32 KB)	
L2 Cache	~12 cycles	(~256 KB - 1 MB)	
L3 Cache	~40 cycles	(a few - tens MB)	
Main Memory	~100-300 cycles (GBs)		slowest

Instruction execution loop (fetch-decode-execute)



SIMD mental model (scalar vs vector)

Scalar (1 add per instr):	Vector/SIMD (8 adds per instr):
a0 + b0 = c0	[a0 a1 a2 a3 a4 a5 a6 a7]
a1 + b1 = c1	+ [b0 b1 b2 b3 b4 b5 b6 b7]
... (8 instructions)	= [c0 c1 c2 c3 c4 c5 c6 c7] (1 instruction)

6. Common Interview Questions

Q1. What is the difference between a register and memory?

- **Answer:** Registers are tiny, ultra-fast storage inside the CPU accessed in ~1 cycle; there are only a few dozen. Memory (RAM) is large (GBs) but ~100+ cycles away. The ALU operates on registers, so data must be loaded from memory into registers to be computed on, then stored back.
- **Key points:** Speed hierarchy, count, load/store model.
- **Common mistake:** Saying "registers are just fast memory" without mentioning they're addressed by name (not by address) and are architecturally special.

Q2. What exactly happens when you call a function?

- **Answer:** Caller places arguments in the convention's registers/stack, executes `call` (pushes the return address, jumps). The callee runs a prologue (saves `rbp`, allocates locals, saves callee-saved regs it uses), does its work, puts the return value in `rax`, runs an epilogue (restores regs), then `ret` pops the return address and jumps back.
- **Key points:** `call/ret`, return address on stack, prologue/epilogue, return value in `rax`.
- **Common mistake:** Forgetting the return address is pushed by `call`, or thinking arguments are always on the stack (they're mostly in registers on x86-64).

Q3. Difference between caller-saved and callee-saved registers?

- **Answer:** Caller-saved registers may be destroyed by a called function, so if the caller needs them after the call it must save them first. Callee-saved registers must be preserved by the callee (saved on entry, restored before return). It's a division that minimizes total save/restore work.
- **Key points:** Who's responsible, why the split exists.
- **Common mistake:** Swapping the definitions, or not being able to name examples (**rax** caller-saved, **rbx/r12-r15** callee-saved on SysV).

Q4. Why does the stack grow downward, and what is a stack frame?

- **Answer:** By convention on x86-64/ARM, the stack grows from high to low addresses (**push** decrements **rsp**). A stack frame is the per-call region holding the return address, saved registers, and locals. Frames chain together via saved base pointers, enabling backtraces.
- **Common mistake:** Confusing "grows downward" (addresses decrease) with the data structure orientation; or claiming all architectures grow down (it's a convention, not a law).

Q5. What is a calling convention and why standardize it?

- **Answer:** A contract specifying argument/return register usage, stack cleanup, and alignment. Standardizing (an ABI) lets independently compiled modules and libraries call each other correctly.
- **Common mistake:** Confusing calling convention with programming-language syntax; not knowing conventions differ by OS (SysV vs Windows x64).

Q6. What's the difference between x86 and ARM (CISC vs RISC)?

- **Answer:** x86-64 is CISC: variable-length instructions, register-memory ops, many addressing modes. ARM is RISC: fixed-length instructions, load-store (arithmetic only on registers), simpler decoding. ARM uses a link register for returns; x86 pushes the return address to the stack.
- **Common mistake:** Saying "RISC is always faster" - modern x86 cores internally crack CISC ops into micro-ops, so real-world performance is comparable; the trade-offs are about decoding, power, and code density.

Q7. What is a system call and how does it work?

- **Answer:** A request from user space into the kernel for a privileged operation. You load a syscall number and args into registers, execute a trap instruction (**syscall/svc**), which switches to kernel mode; the kernel validates and performs the operation, then returns to user mode with the result in a register.
- **Key points:** User→kernel mode switch, trap instruction, number+args in registers.
- **Common mistake:** Confusing a system call (into the OS kernel) with a regular function call or a library call (**printf** is a library call that *eventually* makes a syscall).

Q8. What is SIMD and when is it useful?

- **Answer:** Single Instruction Multiple Data - one instruction processes a vector of elements (e.g. 8 floats). Great for data-parallel workloads: multimedia, ML, crypto, parsing. Requires data to be laid out contiguously and ideally aligned.
- **Common mistake:** Confusing SIMD (data parallelism within a core) with multithreading (task parallelism across cores).

Q9. What causes a stack overflow at the assembly level?

- **Answer:** The stack keeps growing (each call subtracts from `rsp`) until it hits the stack's guard page / limit. Unbounded recursion or huge local arrays exhaust it, triggering a fault.
- **Common mistake:** Confusing stack overflow (runtime stack exhaustion) with a heap overflow or with the website Stack Overflow.

Q10. How would you see the assembly a compiler produces?

- **Answer:** `gcc -S file.c` emits `.s` assembly; `objdump -d binary` disassembles; `gdb` with `disassemble`; or paste into Godbolt (Compiler Explorer). Use `-O2` to see optimized output.
- **Common mistake:** Only knowing one tool, or forgetting that optimization level dramatically changes the output.

Q11. Why is `xor eax, eax` preferred over `mov eax, 0`?

- **Answer:** It's a smaller instruction, the CPU recognizes it as a zeroing idiom (handled by register renaming with zero latency), and it breaks false dependency chains on the old value of `eax`.
- **Common mistake:** Saying "they're identical" - functionally yes, but `xor` is a known optimization.

Q12. What does the `ret` instruction do, and what's the danger if the stack is corrupted?

- **Answer:** `ret` pops the top of the stack (the saved return address) into `rip` and jumps there. If an attacker overwrites that saved address (e.g. via buffer overflow), `ret` jumps to attacker-controlled code - the basis of stack-smashing and ROP attacks.
- **Common mistake:** Not connecting `ret` to security (return-oriented programming, control-flow hijacking).

7. Deep-Dive Questions

D1. How does the red zone work in the System V x86-64 ABI, and why do leaf functions benefit? The **red zone** is 128 bytes below `rsp` that a function may use without adjusting `rsp`. **Leaf functions** (which call no other function) can use it for locals without a prologue that moves `rsp`, saving instructions. It's safe because nothing (in user code) will clobber below `rsp`... except signal handlers, which is why the kernel and signal-handling code don't rely on the red zone (`-mno-red-zone` in kernels).

D2. What is stack alignment and why must `rsp` be 16-byte aligned at a `call`? The ABI requires `rsp` to be 16-byte aligned *before* a `call` (so it's `16n+8` on function entry after the return address is pushed). This is because SIMD instructions (`movaps`) require 16-byte-aligned memory, and functions assume alignment for their locals. Misalignment causes crashes or slow unaligned accesses. Compilers insert `sub rsp, 8` padding to maintain it.

D3. How does branch prediction interact with speculative execution, and how did that lead to Spectre/Meltdown? CPUs predict branch outcomes and **speculatively execute** ahead. If mispredicted, results are discarded architecturally - but micro-architectural state (cache) changes remain. **Spectre** tricks the predictor into speculatively accessing secret-dependent memory, then leaks it via a cache-timing side channel. Mitigations include `lfence` barriers, retpolines, and microcode updates. This is why understanding speculation is now a security topic.

D4. How do position-independent code (PIC) and the GOT/PLT work for calling shared library functions? PIC lets code run at any load address (needed for ASLR and shared libraries). Calls to external

functions go through the **PLT** (Procedure Linkage Table), which on first call uses the dynamic linker to resolve the real address and caches it in the **GOT** (Global Offset Table). Subsequent calls jump directly. This is why `objdump` shows `call foo@plt` and RIP-relative addressing (`lea rax, [rip + offset]`) is pervasive in 64-bit code.

D5. What are the correctness rules for GCC extended inline assembly (outputs, inputs, clobbers, volatile)? You must declare every register/memory the asm reads (inputs), writes (outputs), and destroys (clobber list, including `"cc"` for flags and `"memory"` if it accesses memory the compiler can't see). Omitting a clobber lets the compiler assume a register is untouched and reuse a stale value → miscompilation. `volatile` prevents the compiler from deleting or reordering asm with no visible outputs (like `rdtsc` or a memory barrier). Getting constraints wrong produces bugs that only appear at higher optimization levels.

8. Comparison Tables

Caller-Saved vs Callee-Saved (x86-64 System V)

Aspect	Caller-Saved (volatile)	Callee-Saved (non-volatile)
Who preserves	Caller (before making a call)	Callee (before using them)
Callee may overwrite?	Yes, freely	No, must restore
Typical use	Short-lived scratch values	Values kept across calls
Examples (SysV)	<code>rax, rcx, rdx, rsi, rdi, r8-r11</code>	<code>rbx, rbp, r12-r15, rsp</code>
Cost paid when	Value needed after a call	Register actually used

x86-64 (CISC) vs ARM64 (RISC)

Feature	x86-64	ARM64 (AArch64)
ISA type	CISC	RISC
Instruction length	Variable (1-15 bytes)	Fixed (4 bytes)
Memory in arithmetic	Yes (register-memory)	No (load-store only)
General registers	16	31
Return address	Pushed on stack by <code>call</code>	Saved in link register (<code>x30</code>) by <code>bl</code>
Addressing modes	Many, complex	Fewer, simpler
Typical use	Desktops, servers	Mobile, Apple Silicon, embedded, servers

Load-Store vs Register-Memory Architecture

Aspect	Load-Store (ARM, RISC-V, MIPS)	Register-Memory (x86)
Arithmetic operands	Registers only	Registers or memory
Memory access	Explicit <code>ldr/str</code>	Implicit in many ops

Aspect	Load-Store (ARM, RISC-V, MIPS)	Register-Memory (x86)
Instruction count	More (must load first)	Fewer per operation
Decoding	Simpler, uniform	More complex

System Call vs Function Call vs Library Call

Aspect	Function Call	Library Call	System Call
Crosses privilege boundary	No	No	Yes (user→kernel)
Mechanism	<code>call/ret</code>	<code>call/ret</code> (into libc)	<code>syscall/svc</code> trap
Cost	~cycles	~cycles	~hundreds of cycles (mode switch)
Example	your <code>add()</code>	<code>printf()</code>	<code>write()</code>

SSE vs AVX vs AVX-512 (x86 SIMD)

Feature	SSE	AVX/AVX2	AVX-512
Register width	128-bit (<code>xmm</code>)	256-bit (<code>ymm</code>)	512-bit (<code>zmm</code>)
Floats per op (32-bit)	4	8	16
Introduced	~1999	2011/2013	2016
Note	Baseline on all x86-64	Common on modern CPUs	Server/high-end; can cause frequency throttling

Intel vs AT&T Syntax

Aspect	Intel	AT&T
Operand order	<code>mov dst, src</code>	<code>movq %src, %dst</code>
Registers	<code>rax</code>	<code>%rax</code>
Immediates	<code>mov rax, 5</code>	<code>mov \$5, %rax</code>
Memory	<code>[rbx+4]</code>	<code>4(%rbx)</code>
Used by	Intel docs, NASM, MSVC	GNU <code>as</code> , default <code>objdump/gcc -S</code>

9. Common Mistakes

- **"Registers are just fast RAM."** They're architecturally named, finite, and the only operands the ALU touches directly - a different category from addressable memory.
- **Confusing the stack (data structure) with the stack (memory region).** The call stack is a specific downward-growing memory region managed by `rsp`.

- **Thinking arguments are always passed on the stack.** On x86-64 and ARM64, the first several arguments go in registers; only overflow spills to the stack.
- **Swapping caller-saved and callee-saved.** Mnemonic: *callee-saved = the callee saves it for you; it's preserved across the call.*
- **Assuming one calling convention everywhere.** Linux/macOS (SysV) and Windows x64 differ in argument registers and shadow space.
- **Confusing a library call with a system call.** `printf` is a libc function; it internally issues the `write` syscall. Not every function call enters the kernel.
- **Believing "stack grows up."** On mainstream architectures it grows toward lower addresses.
- **Confusing SIMD with multithreading.** SIMD = one core, multiple data lanes; threads = multiple cores/contexts.
- **Writing inline asm without a clobber list.** Undeclared clobbers cause silent miscompilation, often only at `-O2`.
- **Reading AT&T syntax as Intel.** The operand order is reversed - a frequent source of confusion when comparing `objdump` output to Intel manuals.
- **Ignoring 32-bit-write zero-extension on x86-64.** `mov eax, ...` clears the upper 32 bits of `rax`; `mov al, ...` does not clear the rest.

10. Edge Cases / Special Cases

- **32-bit writes zero-extend, 8/16-bit writes don't.** `mov eax, 1` → `rax = 1`. But `mov al, 1` leaves bits 8-63 of `rax` unchanged (creating partial-register stalls).
- **The red zone (128 bytes below `rsp`)** is usable by leaf functions without adjusting `rsp` - but it's clobbered by signal handlers and unavailable in kernel code.
- **16-byte stack alignment at `call` sites** is mandatory; violating it crashes on aligned SIMD moves.
- **`div`/`idiv` use implicit registers.** x86 division divides the 128-bit `rdx:rax` by the operand, putting quotient in `rax`, remainder in `rdx`. Forgetting to zero/sign-extend `rdx` first gives wrong results or a `#DE` exception (also on divide-by-zero).
- **Signed vs unsigned branches.** After `cmp`, use `jg/jl` (signed) vs `ja/jb` (unsigned). Mixing them is a classic bug.
- **Leaf vs non-leaf functions.** A leaf function calling nothing can skip frame setup and use scratch registers freely.
- **Tail calls.** A `jmp` (instead of `call`) to another function reuses the current frame - important for recursion and functional-language performance.
- **`nop` and instruction padding** are used for alignment and hot-patching, not just "do nothing".
- **AVX-512 frequency throttling.** Heavy AVX-512 use can lower the CPU clock, sometimes making narrower SIMD faster overall.
- **Endianness.** x86 and ARM (usually) are little-endian; bytes in memory are least-significant-first. Matters when reinterpreting memory or in network code.
- **Delay slots** exist on some RISC ISAs (classic MIPS): the instruction after a branch always executes. AArch64 does **not** have them.

11. How to Explain in Interview

"At the lowest level, a CPU only works with a small set of super-fast **registers** and can only compute on values that are in them, so code is a stream of load, compute, and store steps plus jumps for control

flow. Function calls work through a **calling convention**: arguments go in specific registers, `call` pushes a return address and jumps, the function sets up a **stack frame** for its locals, does its work, puts the result in `rax` (or `x0` on ARM), restores the **callee-saved** registers it touched, and `rets` back. x86-64 is CISC and can operate on memory directly; ARM is RISC and load-store, so it must move data into registers first. When a program needs the OS - to read a file or write to the screen - it issues a **system call** that traps into the kernel. And when I need raw throughput, **SIMD** lets one instruction process a whole vector of data at once. I use `gcc -S` or Godbolt to see exactly what the compiler emits."

Keep it to that arc: *registers & load/store* → *function calls & stack frames* → *conventions & saved registers* → *x86 vs ARM* → *syscalls* → *SIMD* → *tooling*.

12. Quick Revision Notes

Key definitions

- **Register**: named, ~1-cycle CPU storage; only operands the ALU uses directly.
- **Load/Store**: memory→register / register→memory.
- **Stack frame**: per-call region with return address, saved regs, locals.
- **Calling convention (ABI)**: contract for args, return values, saved regs, alignment.
- **System call**: trap into the kernel for a privileged operation.
- **SIMD**: one instruction, many data lanes.

Important points

- ALU works on registers, not memory (fully true for RISC; x86 allows one memory operand).
- Stack grows **downward**; `rsp` = top, `rbp` = frame base.
- `call` pushes return address; `ret` pops it. ARM uses link register `x30`.
- SysV integer args: `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`; return in `rax`.
- Callee-saved: `rbx`, `rbp`, `r12-r15`. Caller-saved: `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8-r11`.
- `rsp` must be 16-byte aligned at a `call`.
- `xor reg, reg` is the fast zeroing idiom.

Common comparisons: caller vs callee-saved; x86 (CISC) vs ARM (RISC); load-store vs register-memory; function vs library vs system call; SSE vs AVX vs AVX-512.

Must-remember facts

- Writing `eax` zero-extends into `rax`; writing `al` doesn't.
- `printf` is a library call; `write` is the syscall underneath.
- SIMD ≠ threads (data parallelism vs task parallelism).
- Overwriting the saved return address → control-flow hijack (ROP).

Interview traps

- Don't swap caller/callee-saved.
- Don't say the stack grows up.
- Don't confuse library and system calls.
- Don't forget clobber lists in inline asm.
- Don't mix signed (`jg/jl`) and unsigned (`ja/jb`) branches.

13. Practice Tasks

1. **Read compiler output.** Write `int square(int x){return x*x;}` and run `gcc -O2 -S square.c`. Identify the argument register and the return register.
2. **Trace a call by hand.** For a function `f` that calls `g`, draw the stack: return address, saved `rbp`, locals. Show what `push rbp; mov rbp, rsp` does.
3. **Disassemble a binary.** Compile any small C program and run `objdump -d a.out`. Find `main`, the prologue, and the `call` to `printf`.
4. **Hand-write a Linux syscall in assembly.** Write a NASM program that writes "Hello\n" to stdout and `exits`, using `syscall`. Assemble and run it.
5. **Compare architectures on Godbolt.** Paste the same C function and switch the compiler between x86-64 gcc and aarch64 gcc. List three differences you see.
6. **Prove caller/callee-saved behavior.** Write a function with a loop that calls another function each iteration, keeping a counter across the call. Check whether the compiler placed the counter in `rbx/r12-r15` (callee-saved) - and reason why.
7. **Strength reduction.** Compile `x * 8, x / 2, x % 4` at `-O2` and confirm the compiler used shifts/`and` instead of `mul/div`.
8. **SIMD experiment.** Write a loop summing a float array; compile with `-O3 -march=native` and check for `vaddps/addps`. Then try `-fno-tree-vectorize` and compare.
9. **Inline assembly.** Implement `rdtsc()` using GCC extended asm and measure how many cycles a loop takes.
10. **Branch vs branchless.** Implement `max(a,b)` with a branch and with `cmov` (or the compiler's branchless form) and compare the disassembly.
11. **Stack overflow.** Write infinite recursion, run it, and observe the crash. Then inspect the backtrace in `gdb` to see the repeated frames.
12. **Signed/unsigned trap.** Compile a comparison of `int` vs `unsigned` values and note whether the compiler emitted `j1` or `jb`.

14. Final Cheat Sheet

```
===== ASSEMBLY & LOW-LEVEL =====
```

CORE DEFINITION

```
Assembly = human-readable machine instructions. CPU computes on fast
REGISTERS; moves data to/from MEMORY via LOAD/STORE; jumps for control
flow; uses the STACK to remember calls.
```

WHY IT MATTERS

```
Performance, debugging, security, and understanding what abstractions
really do. Foundation of OS, compilers, and systems work.
```

THE EXECUTION MODEL

```
fetch -> decode -> execute -> writeback ; rip/pc advances
ALU works on registers only (RISC) / mostly registers (x86 allows 1 mem).
```

KEY REGISTERS (x86-64)

```

rax=return  rsp=stack ptr  rbp=frame base  rip=instr ptr
args: rdi rsi rdx rcx r8 r9      (SysV Linux/macOS)
args: rcx rdx r8 r9 + shadow    (Windows x64)
ARM64: x0..x30, x0=return, x30=LR, sp, pc ; bl=call, ret via LR

```

SAVED REGISTERS (SysV)

```

caller-saved: rax rcx rdx rsi rdi r8-r11  (callee may trash)
callee-saved: rbx rbp r12-r15 rsp         (callee must restore)

```

FUNCTION CALL

```

caller: put args in regs -> call (pushes return addr)
callee: push rbp; mov rbp, rsp; sub rsp, N ; ...; leave; ret

```

INSTRUCTION FAMILIES

```

data:  mov / ldr, str      arith: add sub imul idiv
logic: and or xor shl sar test cmp
ctrl:  jmp je jne jg/jl(signed) ja/jb(unsigned) call ret

```

SYSTEM CALL (Linux x86-64)

```

rax=number, args in rdi rsi rdx r10 r8 r9, then `syscall`
(ARM64: x8=number, args x0.., then `svc #0`)

```

SIMD

```

x86: xmm(128)/ymm(256)/zmm(512), e.g. vaddps ; ARM: NEON/SVE
one instruction, many lanes ; data parallelism, not threads

```

TOOLS

```

gcc -S file.c          # emit assembly
objdump -d bin         # disassemble
gdb / radare2 / Ghidra / godbolt.org

```

MOST ASKED QUESTIONS

1. Register vs memory?
2. What happens on a function call?
3. Caller-saved vs callee-saved?
4. x86 (CISC) vs ARM (RISC)?
5. What is a system call?
6. What is SIMD?
7. What is a stack frame / stack overflow?

TOP COMPARISONS

```

caller vs callee-saved | CISC vs RISC | load-store vs register-memory
function vs library vs system call | SSE vs AVX vs AVX-512

```

ONE-LINE INTERVIEW ANSWER

```

"A CPU only computes on a few fast registers, so programs load data in,
compute, store it back, and jump around; function calls follow a
calling convention using the stack, and syscalls trap into the kernel
for privileged work."

```

```
=====
```